

Semáforos

Limitación soluciones previas

- Los algoritmos de la clase anterior asumen lectura y escritura atómica (máquina básica).
- Soluciones **ineficientes** para un uso práctico.

Sincronización de más alto nivel

Limitación soluciones previas

- Los algoritmos de la clase anterior asumen lectura y escritura atómica (máquina básica).
- Soluciones **ineficientes** para un uso práctico.

Más alto nivel: Semáforos

Tipo de datos para programación concurrente:

- Implementado generalmente por el **Sistema Operativo**.
- Concepto “simple” y ampliamente utilizado
- Son todavía de bajo nivel: propenso a errores.

Asignación de Hardware

- **Multiprocesamiento Puro:** Procesadores $\geq$ Procesos. Cada proceso dispone de una CPU dedicada de forma permanente.
- **Multitarea (Multiplexación):** Procesos $>$ Procesadores. Los procesos deben competir y compartir el tiempo de CPU.

El Planificador y el Modelo de Concurrency

El Planificador (Scheduler)

Programa del sistema encargado de:

- Decidir qué proceso listo pasa a ejecución.
- Realizar el **cambio de contexto** (context switch).

El Planificador y el Modelo de Concurrency

El Planificador (Scheduler)

Programa del sistema encargado de:

- Decidir qué proceso listo pasa a ejecución.
- Realizar el **cambio de contexto** (context switch).

Nuestro Modelo de Entrelazado

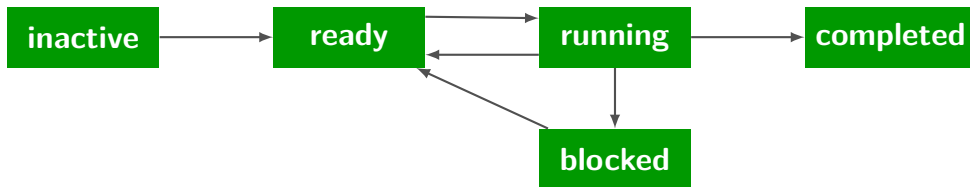
- Para simplificar el modelo, no mencionamos explícitamente al scheduler.
- Asume un entrelazado **arbitrario** de sentencias atómicas.
- El scheduler puede hacer un cambio de contexto en **cualquier momento** (incluso tras una sola instrucción).

Estados de un proceso

Estados

- **Ejecución (running):** El proceso tiene asignada una CPU y está ejecutando activamente sus instrucciones.
- **Listo (ready):** El proceso posee todos los recursos necesarios para ejecutar, pero espera por la CPU.
- **Bloqueado / Espera (blocked/waiting):** El proceso no puede ejecutar porque está esperando un evento externo (operación de E/S).
- **Inactivo (inactive):** No está cargado en memoria principal.
- **Terminado (completed):** Terminó su ejecución.

Ciclo de vida



Notación

- Dado un proceso p , $p.state$ indica su estado actual.
- Usamos asignación para representar cambios de estado:

`p.state := ready`

`p.state := running`

Semáforo: definición

Semáforo

Semáforo: definición

Semáforo

Un tipo de dato abstracto que representa enteros no negativos.

Semáforo: definición

Semáforo

Un tipo de dato abstracto que representa enteros no negativos.

Operaciones

- **wait (originalmente P):** Decrementa el valor. Si es cero, la operación se bloquea hasta que el valor sea positivo. .
- **signal (originalmente V):** Incrementa el valor. Si operaciones wait bloqueads se desbloquea una de ella.

Semáforo: Representación

Representación

```
int V; // Valor  
conjunto<procesos> L; //procesos esperando ejecutar un wait
```

Semáforo: Representación

Representación

```
int V; // Valor  
conjunto<procesos> L; //procesos esperando ejecutar un wait
```

Semáforo: Representación

Representación

```
int V; // Valor  
conjunto<procesos> L; //procesos esperando ejecutar un wait
```

Invariante de Representación

- $V \geq 0$.
- $V > 0 \implies L = \emptyset$.

Como corolario, tenemos que $L \neq \emptyset \implies V = 0$.

Semáforo: Implementación de Operaciones

Semáforo: Implementación de Operaciones

wait

```
wait(){  
  if this.V > 0 then  
    this.V := this.V - 1  
  else  
    this.L := this.L U {p}  
    p.state := bloqueado  
}
```

signal

```
signal(){  
  if this.L = {} then  
    this.V := this.V + 1  
  else  
    let p in this.L  
    this.L := this.L - {p}  
    p.state := ready  
}
```

- p denota al proceso que invoca a la operación wait.
- Las dos operaciones son **atómicas**.

Semáforo: Constructor

Constructor

```
Semáforo(int k) {  
    //pre: k >= 0  
    this.V := k  
    this.L := {}  
}
```

Teorema $S = \text{Semáforo}(k)$

- a. $S.V \geq 0$
- b. $S.V = k + \#signal - \#wait$

- k : Valor inicial de $S.V$.
- $\#signal$ y $\#wait$: Cantidad de operaciones $S.signal()$ y $S.wait()$ completadas.
- Se considera que un proceso bloqueado en un $wait$ no realizó la operación $wait$.

Demostración: Invariantes de Semáforo

Demostración por Inducción estructural

1. Base(Constructor):

Demostración: Invariantes de Semáforo

Demostración por Inducción estructural

1. **Base(Constructor):** $S = \text{Semáforo}(k)$ con $k \geq 0$.

Demostración: Invariantes de Semáforo

Demostración por Inducción estructural

1. **Base(Constructor):** $S = \text{Semáforo}(k)$ con $k \geq 0$.

- $\#wait = 0, \#signal = 0$
- $S.V = k \geq 0 \quad \checkmark(a)$
- $S.V = k + 0 - 0 \quad \checkmark(b)$

Demostración: Invariantes de Semáforo

Demostración por Inducción estructural

1. **Base(Constructor):** $S = \text{Semáforo}(k)$ con $k \geq 0$.

- $\#wait = 0, \#signal = 0$
- $S.V = k \geq 0 \quad \checkmark(a)$
- $S.V = k + 0 - 0 \quad \checkmark(b)$

2. **Caso inductivo:**

Demostración: Invariantes de Semáforo

Demostración por Inducción estructural

1. **Base(Constructor):** $S = \text{Semáforo}(k)$ con $k \geq 0$.

- $\#wait = 0, \#signal = 0$
- $S.V = k \geq 0 \quad \checkmark(a)$
- $S.V = k + 0 - 0 \quad \checkmark(b)$

2. **Caso inductivo:** Assumimos $S.V \geq 0$ y $S.V = k + \#signal - \#wait$.

Caso $signal(S)$:

- Siempre incrementa $\#signal$ en 1.
- Si $S.L = \emptyset$: $S.V$ aumenta en 1. Ambos lados de (b) se incrementan. $S.V \geq 0$ se mantiene.
- Si $S.L \neq \emptyset$: $S.V$ no cambia, pero un proceso sale de $S.L$ (pasa a ready).

Nota: Por definición, esto equivale a que se complete un wait previo, compensando la ecuación (b).

(continua)

Demostración: Invariantes de Semáforo

Demostración por Inducción estructural (cont)

2. **Caso inductivo:** Assumimos $S.V \geq 0$ y $S.V = k + \#signal - \#wait$.

...

Caso wait(S):

- Si $S.V = 0$,
 - El proceso pasa a bloqueado ingresando a S.L.
 - **Efecto:** No cambia S.V (sigue siendo 0) y no se incrementa $\#wait$.
 - No modifica ninguna variable de las ecuaciones, los invariantes (a) y (b) valen por hipótesis inductiva.
- Si $S.V \geq 0$
 - Decrementa S.V en 1 e incrementa $\#wait$ en 1.
 - Al restar 1 a ambos lados, (b) se mantiene. Como $S.V > 0$, al restar 1 queda $S.V \geq 0$ (a).

Semáforo binario (o mutex)

Semáforo Binario

Un semáforo donde V puede tomar valores 0 o 1 (true, false).

Semáforo binario (o mutex)

Semáforo Binario

Un semáforo donde V puede tomar valores 0 o 1 (true, false).

- El constructor tiene como precondition $k \in \{0, 1\}$.
- `wait` no cambia y `signal` se redefine como sigue:

Semáforo binario (o mutex)

Semáforo Binario

Un semáforo donde V puede tomar valores 0 o 1 (true, false).

- El constructor tiene como precondition $k \in \{0, 1\}$.
- `wait` no cambia y `signal` se redefine como sigue:

signal (semáforo binario)

```
signal() {  
    if this.V = 1 then undefined  
    if this.L = {} then  
        this.V := this.V + 1  
    else  
        let p in this.L  
        this.L := this.L - {p}  
        p.state := ready  
}
```

Exclusión mutua con mutex

Usar un semáforo binario $S = \text{SemáforoBinario}(1)$ (inicializado en 1)

ThreadId = 0

```
while(true) {  
    // sección no crítica  
    S.wait()  
    // sección crítica  
    S.signal()  
}
```

ThreadId = 1

```
while(true) {  
    // sección no crítica  
    S.wait()  
    // sección crítica  
    S.signal()  
}
```

Exclusión mutua con mutex (simplificación)

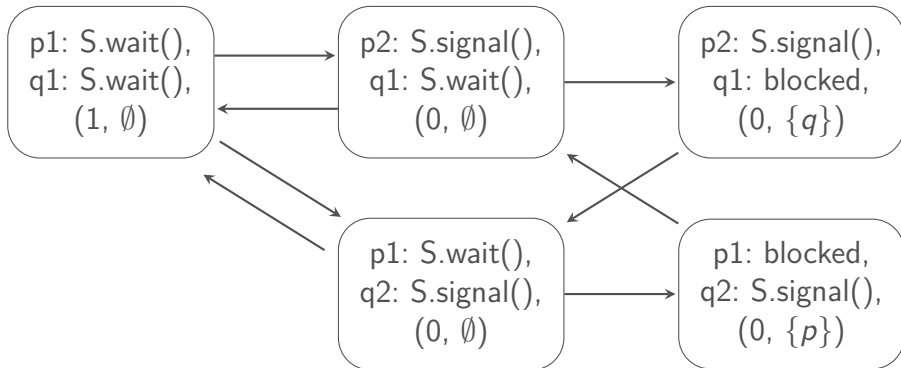
ThreadId = 0

```
while(true) {  
p1:    S.wait()  
p2:    S.signal()  
}
```

ThreadId = 1

```
while(true) {  
q1;    S.wait()  
q2:    S.signal()  
}
```

Grafo de Estados del Semáforo



Propiedades del Grafo de Estados

Exclusión Mutua

Propiedades del Grafo de Estados

Exclusión Mutua

- Una violación requiere un estado de la forma (p2: S.signal(), q2: S.signal(), ...).
- **Ningún estado cumple esa condición.**

Propiedades del Grafo de Estados

Exclusión Mutua

- Una violación requiere un estado de la forma (p2: S.signal(), q2: S.signal(), ...).
- **Ningún estado cumple esa condición.**

Ausencia de Deadlock

Propiedades del Grafo de Estados

Exclusión Mutua

- Una violación requiere un estado de la forma (p2: S.signal(), q2: S.signal(), ...).
- **Ningún estado cumple esa condición.**

Ausencia de Deadlock

- No existen estados en los que **ambos procesos están bloqueados** simultáneamente.

Propiedades del Grafo de Estados

Exclusión Mutua

- Una violación requiere un estado de la forma (p2: S.signal(), q2: S.signal(), ...).
- **Ningún estado cumple esa condición.**

Ausencia de Deadlock

- No existen estados en los que **ambos procesos están bloqueados** simultáneamente.

Garantía de entrada

Propiedades del Grafo de Estados

Exclusión Mutua

- Una violación requiere un estado de la forma (p2: `S.signal()`, q2: `S.signal()`, ...).
- **Ningún estado cumple esa condición.**

Ausencia de Deadlock

- No existen estados en los que **ambos procesos están bloqueados** simultáneamente.

Garantía de entrada

- Al salir de la sección no crítica (`wait`), un proceso va al estado `signal` (sección crítica) o se bloquea.
- La única transición de salida de un estado `blocked` va al estado donde el proceso entra a la sección crítica (`signal`).

Correctitud de la solución (2 procesos)

Invariantes:

- a. $\#CS = \#wait(S) - \#signal(S)$
- b. $\#CS + S.V = 1$
 - $\#CS$: Cantidad de procesos ejecutando su región crítica.

Correctitud de la solución (2 procesos)

Invariantes:

- a. $\#CS = \#wait(S) - \#signal(S)$
- b. $\#CS + S.V = 1$
 - $\#CS$: Cantidad de procesos ejecutando su región crítica.

Prueba:

- a. Por inducción en la longitud de la traza de ejecución.

Correctitud de la solución (2 procesos)

Invariantes:

- a. $\#CS = \#wait(S) - \#signal(S)$
- b. $\#CS + S.V = 1$
 - $\#CS$: Cantidad de procesos ejecutando su región crítica.

Prueba:

- a. Por inducción en la longitud de la traza de ejecución.
- b. Usando el invariante propio del semáforo
($S.V = k + \#signal(S) - \#wait(S)$):

Correctitud de la solución (2 procesos)

Invariantes:

- a. $\#CS = \#wait(S) - \#signal(S)$
- b. $\#CS + S.V = 1$
 - $\#CS$: Cantidad de procesos ejecutando su región crítica.

Prueba:

- a. Por inducción en la longitud de la traza de ejecución.
- b. Usando el invariante propio del semáforo
($S.V = k + \#signal(S) - \#wait(S)$): Como el valor inicial es $k = 1$:

$$S.V = 1 + \#signal(S) - \#wait(S)$$

Sustituyendo por el invariante (a):

$$S.V = 1 - \#CS \implies \#CS + S.V = 1$$

Propiedades de la solución

Notar que $\#CS + S.V = 1$:

- **Garantiza exclusión mutua:**

Propiedades de la solución

Notar que $\#CS + S.V = 1$:

- **Garantiza exclusión mutua:** ($\#CS \leq 1$).

Propiedades de la solución

Notar que $\#CS + S.V = 1$:

- **Garantiza exclusión mutua:** ($\#CS \leq 1$).
- **Garantiza ausencia de deadlocks:**

Propiedades de la solución

Notar que $\#CS + S.V = 1$:

- **Garantiza exclusión mutua:** ($\#CS \leq 1$).
- **Garantiza ausencia de deadlocks:** Dos procesos bloqueados en wait implican $S.V = 0$ y $\#CS = 0$ (Contradicción: $0 + 0 \neq 1$).

Propiedades de la solución

Notar que $\#CS + S.V = 1$:

- **Garantiza exclusión mutua:** ($\#CS \leq 1$).
- **Garantiza ausencia de deadlocks:** Dos procesos bloqueados en wait implican $S.V = 0$ y $\#CS = 0$ (Contradicción: $0 + 0 \neq 1$).
- **Garantiza entrada:** Por absurdo. Suponer que un proceso p está bloqueado (quiere entrar) indefinidamente. Por invariantes, $\#CS = 1 - S.V = 1 - 0 = 1$, por lo tanto, el otro proceso q se encuentra en la sección crítica. Como la sección crítica termina, q completará la sección crítica y ejecutará `signal(S)`. Dado que solo hay dos procesos en el programa, $S.L$ debe ser igual al conjunto $\{p\}$. Luego p se desbloqueará e ingresará a su sección crítica.

Sección crítica para N procesos

Es posible usar la misma solución: un semáforo binario $S = \text{Semáforo}(1)$ (inicializado en 1)

ThreadId = i ($i \in \{0, \dots, N-1\}$)

```
while(true) {  
    // seccion no critica  
    S.wait()  
    // seccion critica  
    S.signal()  
}
```

Sección crítica para N procesos

Es posible usar la misma solución: un semáforo binario $S = \text{Semáforo}(1)$ (inicializado en 1)

ThreadId = i ($i \in \{0, \dots, N-1\}$)

```
while(true) {  
    // seccion no critica  
    S.wait()  
    // seccion critica  
    S.signal()  
}
```

Invariantes:

- a. $\#CS = \#wait(S) - \#signal(S)$
- b. $\#CS + S.V = 1$

Prueba: Análoga a la solución binaria.

Propiedades solución basada en semáforos para N procesos

Notar que $\#CS + S.V = 1$:

- **Garantiza exclusión mutua:** ($\#CS \leq 1$).
- **Garantiza ausencia de deadlocks:** Dos procesos bloqueados en wait implican $S.V = 0$ y $\#CS = 0$ (Contradicción: $0 + 0 \neq 1$).
- **No Garantiza entrada:**

Propiedades solución basada en semáforos para N procesos

Notar que $\#CS + S.V = 1$:

- **Garantiza exclusión mutua:** ($\#CS \leq 1$).
- **Garantiza ausencia de deadlocks:** Dos procesos bloqueados en wait implican $S.V = 0$ y $\#CS = 0$ (Contradicción: $0 + 0 \neq 1$).
- **No Garantiza entrada:** Por absurdo. Suponer que un proceso p está bloqueado (quiere entrar) indefinidamente. Por invariantes, $\#CS = 1 - S.V = 1 - 0 = 1$, por lo tanto, el otro proceso q se encuentra en la sección crítica. Como la sección crítica termina, q completará la sección crítica y ejecutará `signal(S)`. El conjunto de procesos en espera no necesariamente es un singleton. Luego puede despertarse otro proceso.

Traza de Ejecución para 3 Procesos

n	Proceso p	Proceso q	Proceso r	S
1	p1: S.wait()	q1: S.wait()	r1: S.wait()	$(1, \emptyset)$
2	p2: S.signal()	q1: S.wait()	r1: S.wait()	$(0, \emptyset)$
3	p2: S.signal()	q1: blocked	r1: S.wait()	$(0, \{q\})$
4	p1: S.signal()	q1: blocked	r1: blocked	$(0, \{q, r\})$
5	p1: S.wait()	q1: blocked	r2: S.signal()	$(0, \{q\})$
6	p1: blocked	q1: blocked	r2: S.signal()	$(0, \{p, q\})$
7	p2: S.signal()	q1: blocked	r1: S.wait()	$(0, \{q\})$

- Las columnas 3 y 7 coinciden, luego se puede construir una traza infinita en la cual q no puede entrar.
- El problema es la elección del proceso en el conjunto

Propiedades solución basada en semáforos para N procesos

Semáforos fuertes (fair):

- El conjunto de procesos bloqueados se implementa sobre una **cola (FIFO)**.

Propiedades solución basada en semáforos para N procesos

Semáforos fuertes (fair):

- El conjunto de procesos bloqueados se implementa sobre una **cola (FIFO)**.

wait

```
wait() {  
  if S.V > 0 then  
    S.V := S.V - 1  
  else  
    S.L := append(S.L, p)  
    p.state := bloqueado  
}
```

signal

```
signal() {  
  if S.L = {} then  
    S.V := S.V + 1  
  else  
    q := head(S.L)  
    S.L := tail(S.L)  
    q.state := ready  
}
```

Inanición y Semáforos Fuertes

Semáforos Débiles vs. Fuertes

- Con más de dos procesos, la solución estándar puede sufrir de **inanición**.
- Con un **semáforo fuerte**, la inanición es **imposible** para cualquier número N de procesos.

Inanición y Semáforos Fuertes

Semáforos Débiles vs. Fuertes

- Con más de dos procesos, la solución estándar puede sufrir de **inanición**.
- Con un **semáforo fuerte**, la inanición es **imposible** para cualquier número N de procesos.

Prueba de Ausencia de Inanición

Supongamos que un proceso p está bloqueado en S (es decir, $p \in S.L$):

1. Como hay N procesos, la cola $S.L$ puede tener a lo sumo $N - 1$ procesos.
2. Por lo tanto, hay a lo sumo $N - 2$ **procesos por delante** de p en la cola.
3. Después de un máximo de $N - 2$ operaciones $S.\text{signal}()$, p llegará a la cabeza de la cola.
4. En consecuencia, se desbloqueará en la siguiente operación $S.\text{signal}()$, y entrará a su sección crítica.

Otros problemas de sincronización clásicos

- Orden de ejecución
- Filósofos comensales
- Productor/Consumidor
- Lectores/Escritores
- Varios más: Fumadores, barbero, santa Claus, ...

Orden de ejecución

La sincronización se usa para **coordinar el orden** de ejecución entre procesos independientes.

Orden de ejecución

La sincronización se usa para **coordinar el orden** de ejecución entre procesos independientes.

Ejemplo: Mergesort Concurrente

Para ordenar el arreglo [5, 1, 10, 7, 4, 3, 12, 8], dividimos el trabajo en tres procesos:

Distribución de Procesos:

- **Proceso 1 (Sort Izquierdo):** Ordena [5, 1, 10, 7] para obtener [1, 5, 7, 10].
- **Proceso 2 (Sort Derecho):** Ordena [4, 3, 12, 8] para obtener [3, 4, 8, 12].
- **Proceso 3 (Merge):** Combina resultados para obtener el arreglo final.

Orden de ejecución

La sincronización se usa para **coordinar el orden** de ejecución entre procesos independientes.

Ejemplo: Mergesort Concurrente

Para ordenar el arreglo [5, 1, 10, 7, 4, 3, 12, 8], dividimos el trabajo en tres procesos:

Distribución de Procesos:

- **Proceso 1 (Sort Izquierdo):** Ordena [5, 1, 10, 7] para obtener [1, 5, 7, 10].
- **Proceso 2 (Sort Derecho):** Ordena [4, 3, 12, 8] para obtener [3, 4, 8, 12].
- **Proceso 3 (Merge):** Combina resultados para obtener el arreglo final.

Requisito de Sincronización:

- Los procesos de ordenamiento son **independientes**.
- El proceso *Merge* **no debe ejecutarse** hasta que los otros dos hayan terminado completamente.

Algoritmo: Mergesort Concurrente

Variables Globales:

```
int[] A  
Semáforo S1, S2  
S1 = SemáforoBinario(0)  
S2 = SemáforoBinario(0)
```

sortIzq

```
p1: sort izquierda  
p2: S1.signal()
```

sortDer

```
q1: sort derecha  
q2: S2.signal()
```

merge

```
r1: S1.wait()  
r2: S2.wait()  
r3: merge
```

El Problema del Productor-Consumidor

- Un problema de **coordinación de orden de ejecución** con dos tipos de procesos que interactúan a través de un buffer compartido:
 - **Productores:** Ejecutan la acción `produce` para crear un elemento de datos y enviarlo.
 - **Consumidores:** Reciben el elemento y ejecutan `consume(dato)` utilizándolo como parámetro.

El Problema del Productor-Consumidor

- Un problema de **coordinación de orden de ejecución** con dos tipos de procesos que interactúan a través de un buffer compartido:
 - **Productores:** Ejecutan la acción produce para crear un elemento de datos y enviarlo.
 - **Consumidores:** Reciben el elemento y ejecutan consume(dato) utilizándolo como parámetro.

Problemas de Sincronización

1. **Buffer Vacío:** Un consumidor no puede extraer un elemento si no hay nada guardado.
2. **Buffer Lleno:** Un productor no puede agregar elementos si está lleno.

Tipos de Buffer

- **Buffer acotado:** El productor tiene que esperar cuando el buffer está lleno; el consumidor espera cuando está vacío.
- **Buffer no acotado:** El productor puede trabajar libremente; el consumidor debe esperar al productor.

Tipos de Buffer

- **Buffer acotado:** El productor tiene que esperar cuando el buffer está lleno; el consumidor espera cuando está vacío.
 - **Buffer no acotado:** El productor puede trabajar libremente; el consumidor debe esperar al productor.
-
- No existen buffers infinitos. Son una abstracción: si el buffer es muy grande en relación a la tasa de producción, se omite el control de buffer lleno para evitar *overhead* asumiendo el riesgo de pérdida u sobreescritura de datos.

Productor-Consumidor: Buffer Infinito

Sincronizar una interacción: **el consumidor no debe extraer un elemento si el buffer está vacío.**

Variables globales

```
ColaInfinita<<Tipo>> buffer := new ... //crea cola vacía.  
Semáforo notEmpty = Semaforo(0)
```


Productor-Consumidor: Buffer Infinito

Sincronizar una interacción: **el consumidor no debe extraer un elemento si el buffer está vacío.**

Variables globales

```
ColaInfinita<<Tipo>> buffer := new ... //crea cola vacía.  
Semáforo  notEmpty = Semaforo(0)
```

Productor

```
Dato d  
while(true) {  
  p1: d := produce()  
  p2: append(d, buffer)  
  p3: notEmpty.signal()  
}
```

Consumidor

```
Dato d  
while(true) {  
  q1: notEmpty.wait()  
  q2: d := take(buffer)  
  q3: consume(d)  
}
```

Productor-Consumidor: Buffer Infinito (Abreviado)

Versión abreviada del algoritmo omitiendo las operaciones locales no críticas (produce y consume).

Variables globales

```
ColaInfinita<<Dato>> buffer := new ... //crea cola vacía.  
Semáforo notEmpty = Semaforo(0)
```

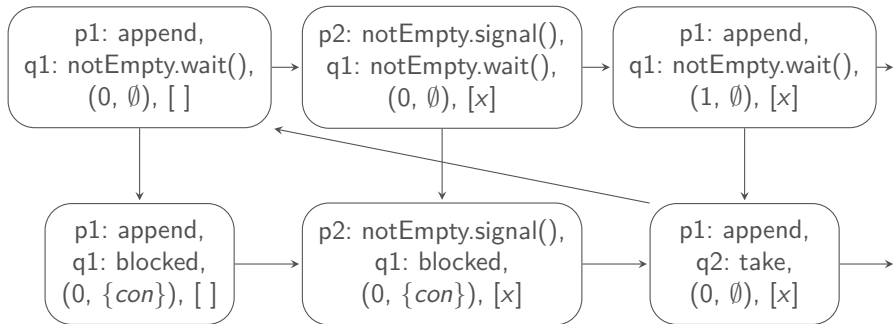
Productor

```
while(true) {  
  p1: append(d, buffer)  
  p2: notEmpty.signal()  
}
```

Consumidor

```
Dato d  
while(true) {  
  q1: notEmpty.wait()  
  q2: d := take(buffer)  
}
```

Grafo de Estados Parcial (Buffer Infinito)



Correctitud: Invariante del Buffer Infinito

Suposición: Cada proceso ejecuta sus dos instrucciones de forma **atómica** (un único paso atómico por ciclo).

Invariante de Conteo

$$\text{notEmpty}.V = \#buffer$$

Donde $\#buffer$ representa la cantidad de elementos presentes en la cola.

Correctitud: Invariante del Buffer Infinito

Suposición: Cada proceso ejecuta sus dos instrucciones de forma **atómica** (un único paso atómico por ciclo).

Invariante de Conteo

$$\text{notEmpty.V} = \# \text{buffer}$$

Donde $\# \text{buffer}$ representa la cantidad de elementos presentes en la cola.

Prueba por Inducción Estructural:

- **Base Inductiva:** Inicialmente, el buffer está vacío ($\# \text{buffer} = 0$) y el semáforo se inicializa en cero ($\text{notEmpty.V} = 0$).

Correctitud: Invariante del Buffer Infinito

Suposición: Cada proceso ejecuta sus dos instrucciones de forma **atómica** (un único paso atómico por ciclo).

Invariante de Conteo

$$\text{notEmpty.V} = \# \text{buffer}$$

Donde $\# \text{buffer}$ representa la cantidad de elementos presentes en la cola.

Prueba por Inducción Estructural:

- **Base Inductiva:** Inicialmente, el buffer está vacío ($\# \text{buffer} = 0$) y el semáforo se inicializa en cero ($\text{notEmpty.V} = 0$).
- **Paso Inductivo (Transiciones):**
 - **Acción del Productor:** Agrega un elemento a la cola ($\# \text{buffer} + 1$) y ejecuta $\text{notEmpty.signal}()$ ($\text{notEmpty.V} + 1$).
 - **Acción del Consumidor:** Extrae un elemento de la cola ($\# \text{buffer} - 1$) al completar un $\text{notEmpty.wait}()$ exitoso ($\text{notEmpty.V} - 1$).

Propiedades de la Solución Abreviada

Ausencia de Deadlock

- Mientras el productor continúe generando elementos de datos de forma activa, ejecutará operaciones `notEmpty.signal()`.
- Estas señales incrementan el valor del semáforo o **desbloquean al consumidor** si este se encontraba suspendido, garantizando que el sistema nunca quede estancado globalmente.

Propiedades de la Solución Abreviada

Ausencia de Deadlock

- Mientras el productor continúe generando elementos de datos de forma activa, ejecutará operaciones `notEmpty.signal()`.
- Estas señales incrementan el valor del semáforo o **desbloquean al consumidor** si este se encontraba suspendido, garantizando que el sistema nunca quede estancado globalmente.

Ausencia de Inanición

- El consumidor es el **único proceso** capaz de ingresar a la cola de espera del semáforo (`notEmpty.L`).
- Al no haber competencia con otros procesos de consumo, la inanición es trivialmente imposible.

¿Por qué funciona si no es atómico?

Si la producción/consumición y las operaciones del semáforo ocurren por separado, el invariante $\text{notEmpty}.V = \#buffer$ deja de cumplirse transitoriamente.

Invariante Relajado

$$\text{notEmpty}.V \leq \#buffer$$

¿Por qué funciona si no es atómico?

Análisis de los Estados Intermedios:

- **En el Productor:** Primero se ejecuta `append` (`#buffer` aumenta) y *luego* `notEmpty.signal()`.
 - En el instante intermedio, hay un elemento real disponible en el buffer que el semáforo aún no contabiliza (`notEmpty.V < #buffer`).
 - **Consecuencia:** Esto es seguro. Lo único que ocurre es que el consumidor podría tener que esperar un instante más.
- **En el Consumidor:** El semáforo decrementa su valor en el `wait()` *antes* de ejecutar la operación `take`.
 - El semáforo reserva el elemento por adelantado. Si el consumidor pasa el `wait`, tiene garantizado que el elemento ya existe en el buffer.

Buffer Finito

Se puede extender el algoritmo para el caso buffer finito:

- El productor **extrae lugares vacíos** del buffer, tal como el consumidor extrae elementos de datos del mismo.
- Usamos un semáforo `notFull` que se inicializa en N , el número de lugares (inicialmente vacíos) en el buffer finito.

Buffer Finito

Se puede extender el algoritmo para el caso buffer finito:

- El productor **extrae lugares vacíos** del buffer, tal como el consumidor extrae elementos de datos del mismo.
- Usamos un semáforo `notFull` que se inicializa en N , el número de lugares (inicialmente vacíos) en el buffer finito.

Dualidad de los Semáforos

- `notEmpty` cuenta la cantidad de elementos de datos disponibles para el consumidor.
- `notFull` cuenta la cantidad de espacios libres disponibles para el productor.

Técnica: Semáforos Partidos (Split Semaphores)

Definición Conceptual

No es un tipo nuevo de semáforo, sino un término de diseño para describir un mecanismo de sincronización específico.

- Consiste en un **grupo de dos o más semáforos** que satisfacen un invariante global.
- La suma de sus componentes enteros es, a lo sumo, igual a un número N .

Semáforos Binarios Partidos (*Split Binary Semaphores*)

Cuando $N = 1$, la estructura se denomina **semáforo binario partido**.

Productor-Consumidor: Buffer de Capacidad 1

Variables globales

```
Dato buffer // Variable compartida unica
Semáforo notEmpty = Semaforo(0) // Indica elemento listo
Semáforo notFull = Semaforo(1) // Indica espacio libre
```

Invariante: $\text{notEmpty.V} + \text{notFull.V} = 1$

Productor

```
Dato d
while(true) {
    d := produce()
    notFull.wait()
    buffer := d
    notEmpty.signal()
}
```

Consumidor

```
Dato d
while(true) {
    notEmpty.wait()
    d := buffer
    notFull.signal()
    consume(d)
}
```

Buffer de tamaño N (Semáforos Generales)

- Implementamos con una cola circular: Un arreglo (buffer) y dos índices (inicio y fin)
- Los productores agregan al frente e incrementan inicio.
- Los consumidores toman de atrás e incrementan fin.

Variables globales

```
Dato[N] buffer
int inicio := 0, fin := 0
Semáforo notFull = Semaforo(N) // N espacios vacios al inicio
Semáforo notEmpty = Semaforo(0) // 0 espacios llenos al inicio
```

Invariante: $\text{notFull.V} + \text{notEmpty.V} \leq N$

Buffer acotado de dimensión N

Variables globales

```
Dato[N] buffer
int inicio := 0, fin := 0
Semáforo notFull = Semaforo(N) // N espacios vacios al inicio
Semáforo notEmpty = Semaforo(0) // 0 espacios llenos al inicio
```

Productor

```
Dato d
while(true) {
    d := produce()
    notFull.wait()
    buffer[inicio] := d
    inicio := (inicio + 1) % N
    notEmpty.signal()
}
```

Consumidor

```
Dato d
while(true) {
    notEmpty.wait()
    d := buffer[fin]
    fin := (fin + 1) % N
    notFull.signal()
    consume(d)
}
```


Falla con Múltiples Consumidores

Asumir quedos consumidores en paralelo acceder a un buffer que contiene dos elementos.

Paso	Consumidor q_1	Consumidor q_2	fin	Buffer
1	notEmpty.wait()	notEmpty.wait()	0	[A, B, ...]
2	d := buffer[fin] (<i>lee A</i>)	notEmpty.wait()	0	[A, B, ...]
3	fin := (fin + 1) % N	notEmpty.wait()	0	[A, B, ...]
4	fin := (fin + 1) % N	d := buffer[fin] (<i>lee A</i>)	0	[A, B, ...]
5	fin := (fin + 1) % N	fin := (fin + 1) % N	1	[A, B, ...]
6	...	fin := (fin + 1) % N	2	[A, B, ...]

Consecuencias

- **Duplicación:** q_1 y q_2 consumieron el mismo elemento (A).
- **Inconsistencia / Pérdida:** El B (ubicado en el índice 1) no se procesará.

Único Productor y Múltiples Consumidores

- Se utiliza un semáforo binario `mutexC` para secuenciar accesos al buffer por parte de los consumidores.
- Valor inicial 1: `mutexC = SemáforoBinario(1)`

Único Productor y Múltiples Consumidores

- Se utiliza un semáforo binario mutexC para secuenciar accesos al buffer por parte de los consumidores.
- Valor inicial 1: mutexC = SemáforoBinario(1)

Productor

```
Dato d
while(true) {
    d := produce()
    notFull.wait()
    buffer[inicio] := d
    inicio := (inicio + 1) % N
    notEmpty.signal()
}
```

Consumidor

```
Dato d
while(true) {
    notEmpty.wait()
    mutexC.wait()
    d := buffer[fin]
    fin := (fin + 1) % N
    mutexC.signal()
    notFull.signal()
    consume(d)
}
```

Multiples Productores y Múltiples Consumidores

Múltiples Productores y Múltiples Consumidores

- Se utiliza un semáforo binario mutexP para secuenciar accesos al buffer por parte de los consumidores.
- Valor inicial 1: mutexP = SemáforoBinario(1)

Productor

```
Dato d
while(true) {
    d := produce()
    notFull.wait()
    mutexP.wait()
    buffer[inicio] := d
    inicio := (inicio + 1) % N
    mutexP.signal()
    notEmpty.signal()
}
```

Consumidor

```
Dato d
while(true) {
    notEmpty.wait()
    mutexC.wait()
    d := buffer[fin]
    fin := (fin + 1) % N
    mutexC.signal()
    notFull.signal()
    consume(d)
}
```

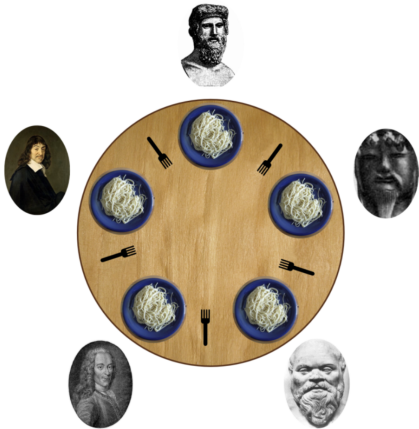
El Problema de los Filósofos Comensales

- Problema clásico usado para comparar formalismos de sincronización.
- Formulación simple, pero solución desafiante.
- 5 filósofos que tienen dos actividades: *pensar* y *comer*.

Dinámica del Filósofo

```
while(true) {  
    p1: pensar  
    p2: preprotocolo  
    p3: comer  
    p4: postprotocolo  
}
```

El Problema de los Filósofos Comensales



El Entorno:

- Mesa con 5 platos, 5 tenedores y una fuente central de spaghetti.
- Para comer, un filósofo **necesita dos tenedores** (el de su izquierda y su derecha).
- Solo puede levantar un tenedor a la vez.

Requisitos de Correctitud del Algoritmo

El diseño de los protocolos (*pre* y *post*) debe garantizar:

- Un filósofo come **solo si** tiene ambos tenedores en su poder.
- **Exclusión Mutua:** No es posible que dos filósofos sostengan el mismo tenedor simultáneamente.
- **Ausencia de deadlock:** El sistema global nunca debe quedar bloqueado o estancado.
- **Ausencia de inanición:** Ningún filósofo debe morir de hambre esperando sus recursos.
- **Eficiencia:** Comportamiento óptimo y rápido en ausencia de contención por los tenedores.

Filósofos comensales: Intento con Array de Semáforos

Variables globales

```
Semáforo tenedores[5] // inicializados en 1
```

Filosofo(i)

```
while(true) {  
    // pensar  
    izq := i  
    der := (i + 1) % 5  
    tenedores[izq].wait()  
    tenedores[der].wait()  
    // comer  
    tenedores[izq].signal()  
    tenedores[der].signal()  
}
```

Filósofos comensales: Intento con Array de Semáforos

Variables globales

```
Semáforo tenedores[5] // inicializados en 1
```

Filosofo(i)

```
while(true) {  
    // pensar  
    izq := i  
    der := (i + 1) % 5  
    tenedores[izq].wait()  
    tenedores[der].wait()  
    // comer  
    tenedores[izq].signal()  
    tenedores[der].signal()  
}
```

No es libre de Deadlock

Puede generar **deadlock**:

1. **Todos** levantan primero su tenedor izquierdo
(`tenedores[izq].wait()`).
2. Quedan **esperando indefinidamente** por su tenedor derecho.
3. Ningún proceso llegará jamás a ejecutar una operación `signal`.

Solución (A): Reducir la Concurrency

Solución (A): Reducir la Concurrency

- No permitir que todos los filósofos intenten tomar tenedores a la vez.
- Usar un semáforo contador sillas con $N - 1$ permisos.

Solución (A): Reducir la Concurrency

- No permitir que todos los filósofos intenten tomar tenedores a la vez.
- Usar un semáforo contador sillas con $N - 1$ permisos.

Variables globales

```
Semáforo tenedores[5]
// inicializados en 1

Semáforo sillas = Semaforo(4)
// N-1 permisos
```

Filosofo(i)

```
while(true) {
    // pensar
    izq := i
    der := (i + 1) % 5
    sillas.wait()
    tenedores[izq].wait()
    tenedores[der].wait()
    // comer
    tenedores[izq].signal()
    tenedores[der].signal()
    sillas.signal()
}
```

Correctitud: Exclusión Mutua de Tenedores

Exclusión Mutua

Ningún tenedor es sostenido por dos filósofos simultáneamente.

Correctitud: Exclusión Mutua de Tenedores

Exclusión Mutua

Ningún tenedor es sostenido por dos filósofos simultáneamente.

Demostración:

- $\#P_i$: cantidad de filósofos que sostienen el tenedor i en un instante dado:

Correctitud: Exclusión Mutua de Tenedores

Exclusión Mutua

Ningún tenedor es sostenido por dos filósofos simultáneamente.

Demostración:

- $\#P_i$: cantidad de filósofos que sostienen el tenedor i en un instante dado:

$$\#P_i = \#wait(tenedores[i]) - \#signal(tenedores[i])$$

Correctitud: Exclusión Mutua de Tenedores

Exclusión Mutua

Ningún tenedor es sostenido por dos filósofos simultáneamente.

Demostración:

- $\#P_i$: cantidad de filósofos que sostienen el tenedor i en un instante dado:

$$\#P_i = \#wait(tenedores[i]) - \#signal(tenedores[i])$$

- Por el invariante propio de los semáforos:

$$tenedores[i].V = k + \#signal(tenedores[i]) - \#wait(tenedores[i])$$

Correctitud: Exclusión Mutua de Tenedores

Exclusión Mutua

Ningún tenedor es sostenido por dos filósofos simultáneamente.

Demostración:

- $\#P_i$: cantidad de filósofos que sostienen el tenedor i en un instante dado:

$$\#P_i = \#wait(tenedores[i]) - \#signal(tenedores[i])$$

- Por el invariante propio de los semáforos:

$$tenedores[i].V = k + \#signal(tenedores[i]) - \#wait(tenedores[i])$$

- Como el valor de inicialización para cada tenedor es $k = 1$:

$$tenedores[i].V = 1 - \#P_i \implies \#P_i + tenedores[i].V = 1$$

Correctitud: Exclusión Mutua de Tenedores

Exclusión Mutua

Ningún tenedor es sostenido por dos filósofos simultáneamente.

Demostración:

- $\#P_i$: cantidad de filósofos que sostienen el tenedor i en un instante dado:

$$\#P_i = \#wait(tenedores[i]) - \#signal(tenedores[i])$$

- Por el invariante propio de los semáforos:

$$tenedores[i].V = k + \#signal(tenedores[i]) - \#wait(tenedores[i])$$

- Como el valor de inicialización para cada tenedor es $k = 1$:

$$tenedores[i].V = 1 - \#P_i \implies \#P_i + tenedores[i].V = 1$$

- Por definición de semáforo **tenedores** $[i].V \geq 0$, se deduce directamente que:

$$\#P_i \leq 1$$

Correctitud: Ausencia de Deadlock

Ausencia de deadlock

El algoritmo es **libre de deadlock**.

Correctitud: Ausencia de Deadlock

Ausencia de deadlock

El algoritmo es **libre de deadlock**.

Demostración por Absurdo:

Correctitud: Ausencia de Deadlock

Ausencia de deadlock

El algoritmo es **libre de deadlock**.

Demostración por Absurdo:

- Suponemos un **deadlock** : **Todos** bloqueados.
- **Análisis de los Recursos (Tenedores):**
 - Por el invariante de sillas, a lo sumo hay **4 filósofos** sentados a la mesa.
 - Cada uno logró ejecutar con éxito su primer `tenedores[izq].wait()`: **cada uno sostiene exactamente 1 tenedor**.
 - En total, los filósofos retienen en sus manos **4 tenedores**.
- **Contradicción:**
 - En la mesa hay **5 tenedores disponibles**.
 - Como solo se están reteniendo 4, **queda 1 tenedor libre en la mesa**.
 - El filósofo que tenga ese tenedor libre a su derecha podrá tomarlo y romperá el ciclo de espera.

Correctitud: Ausencia de Inanición (Parte 1)

Ausencia de Inanición

El algoritmo con restricción de comensales está libre de inanición.

Correctitud: Ausencia de Inanición (Parte 1)

Ausencia de Inanición

El algoritmo con restricción de comensales está libre de inanición.

Premisa: El semáforo `sillas` es fuerte (cola FIFO), por lo que cualquier filósofo que espere para entrar eventualmente lo logrará.

Demostración por Absurdo: Suponemos que el filósofo i está bloqueado para siempre. Por análisis de casos.

- **Caso 1: Bloqueado en su tenedor izquierdo (`tenedores[izq]`).**
 - Esto implica que el vecino de su izquierda ($i - 1$) retiene el recurso como su tenedor derecho.
 - Por la propiedad de **progreso**, el filósofo $i - 1$ eventualmente terminará de comer y ejecutará el `signal`, liberando al filósofo i . ✓

Correctitud: Ausencia de Inanición (Parte 2)

- **Caso 2: Bloqueado en su tenedor derecho (tenedores[der]).**
 - Significa que su vecino derecho ($i + 1$) tomó ese tenedor y nunca lo soltará, quedando a su vez bloqueado eternamente en su propio tenedor derecho.
 - Por inducción, **todos** los filósofos de la mesa tendrían que estar bloqueados en su tenedor derecho simultáneamente.
 - **Contradicción:** Por el invariante de sillas, a lo sumo hay 4 filósofos en la sala. El filósofo ausente no puede estar reteniendo ningún tenedor. ✓

Correctitud: Ausencia de Inanición (Parte 2)

- **Caso 2: Bloqueado en su tenedor derecho (`tenedores[der]`).**
 - Significa que su vecino derecho ($i + 1$) tomó ese tenedor y nunca lo soltará, quedando a su vez bloqueado eternamente en su propio tenedor derecho.
 - Por inducción, **todos** los filósofos de la mesa tendrían que estar bloqueados en su tenedor derecho simultáneamente.
 - **Contradicción:** Por el invariante de sillas, a lo sumo hay 4 filósofos en la sala. El filósofo ausente no puede estar reteniendo ningún tenedor. ✓
- **Caso 3: Bloqueado esperando una silla (`sillas.wait()`).**
 - Esto solo ocurriría si el valor de `sillas.V` fuese cero indefinidamente (sala permanentemente llena con 4 filósofos bloqueados).
 - Sin embargo, los Casos 1 y 2 demostraron que los filósofos que logran ingresar a la sala nunca se bloquean para siempre.
 - Eventualmente, uno terminará de comer y ejecutará `sillas.signal()`, permitiendo el ingreso. ✓

Solución (B): Romper la simetría

- Uno de los filósofos se debe comportar distinto.
- Toma primero el tenedor a su derecha y luego el de su izquierda.

Solución (B): Romper la simetría

- Uno de los filósofos se debe comportar distinto.
- Toma primero el tenedor a su derecha y luego el de su izquierda.

Filosofo(i)

```
if (i == 0) izq = 1; der = 0
else { izq = i; der = (i+1) % N; }
while(true) {
    // pensar
    wait(tenedores[izq]);
    wait(tenedores[der]);
    // comer
    signal(tenedores[izq]);
    signal(tenedores[der]);
}
```

El Problema de los Lectores/Escritores

Es un modelo clásico que abstrae el acceso a una **base de datos o recurso compartido** donde los procesos tienen diferentes intenciones.

El Problema de los Lectores/Escritores

Es un modelo clásico que abstrae el acceso a una **base de datos o recurso compartido** donde los procesos tienen diferentes intenciones.

Lectores (*Readers*)

- Solo consultan o leen la información del recurso.
- No modifican los datos.
- **Concurrencia:** Múltiples lectores pueden acceder **en simultáneo**.

Escritores (*Writers*)

- Modifican o escriben nueva información en el recurso.
- Alteran el estado de los datos.
- **Exclusión:** Requieren acceso **estrictamente exclusivo**.

Restricciones de Sincronización

1. **Escritor vs. Escritor:** Dos escritores no pueden coincidir.
2. **Escritor vs. Lector:** Un escritor no puede ingresar si hay lectores.

Lectores / Escritores (Prioridad Lectores)

Variables globales

Lector

```
while(true) {  
    mutex.wait()  
    #lect := #lect + 1  
    if (#lect == 1) then  
        escribir.wait()  
    mutex.signal()  
    // leer  
    mutex.wait()  
    #lect := #lect - 1  
    if (#lect == 0) then  
        escribir.signal()  
    mutex.signal()  
}
```

Escritor

```
while(true) {  
    escribir.wait()  
    // escribir  
    escribir.signal()  
}
```

Lectores / Escritores (Prioridad Lectores)

Variables globales

```
int #lect := 0
Semáforo mutex = SemaforoBinario(1)
Semáforo escribir = SemaforoBinario(1)
```

Lector

```
while(true) {
    mutex.wait()
    #lect := #lect + 1
    if (#lect == 1) then
        escribir.wait()
    mutex.signal()
    // leer
    mutex.wait()
    #lect := #lect - 1
    if (#lect == 0) then
        escribir.signal()
    mutex.signal()
}
```

Escritor

```
while(true) {
    escribir.wait()
    // escribir
    escribir.signal()
}
```


Variables globales

```
Semáforo escribir = SemaforoBinario(1)
Semáforo leer = SemaforoBinario(1)
Semáforo mutexL = SemaforoBinario(1)
Semáforo mutexE = SemaforoBinario(1),
Semáforo mutex = SemaforoBinario(1)
int #lect := 0, #esc := 0
```

Lectores / Escritores (Prioridad Escritores)

Lector

```
while(true) {  
    mutex.wait()  
    leer.wait()  
    mutexL.wait()  
    #lect := #lect + 1  
    if (#lect == 1) then  
        escribir.wait()  
    mutexL.signal()  
    leer.signal()  
    mutex.signal()  
    // leer  
    mutexL.wait()  
    #lect := #lect - 1  
    if (#lect == 0) then  
        escribir.signal()  
    mutexL.signal()  
}
```

Escritor

```
while(true) {  
    mutexE.wait()  
    #esc := #esc + 1  
    if (#esc == 1) then  
        leer.wait()  
    mutexE.signal()  
  
    escribir.wait()  
    // escribir  
    escribir.signal()  
  
    mutexE.wait()  
    #esc := #esc - 1  
    if (#esc == 0) then  
        leer.signal()  
    mutexE.signal()  
}
```

Semáforos de Espera Activa y Spinlocks

Equivalencia Conceptual

Un **busy-wait semaphore** se comporta exactamente como un **spinlock**:

- En lugar de bloquear al proceso y enviarlo a una cola (*S.L*), el proceso se queda en un bucle infinito (*girando*) evaluando la condición.

Implementación (Spinlock):

```
S.wait() {  
    while (S.V <= 0) {  
        // skip / busy-wait  
    }  
    S.V := S.V - 1  
}
```

Características clave:

- **Consumo de CPU:** Desperdicia ciclos de procesador de forma constante mientras espera.
- **Sin cambio de contexto:** Evita el costo de suspender y reactivar el hilo.
- **Uso ideal:** Sistemas multiprocesador con esperas